Universidad Politécnica de Querétaro

Ingeniería en sistemas computacionales

Materia: Tecnologías y aplicaciones en internet

“MACUIN - Entrega primer parcial”

Docente: Iván Isay Guerra López

Por: Andrés Joshua León Barranco, Jorge Armando López Morales, María de los Ángeles Palma Torres, Santiago Lugo Sánchez

Grupo: S-204

22/ene/2026

Índice de contenido

1. Análisis

1.1. Descripción de la solución planeada

1.2. Diagrama de flujo de la solución

1.3. Requerimientos funcionales

1.4. Requerimientos no funcionales

2. Diseño arquitectónico

2.1. Diagramas de casos de uso

2.2. Diagramas de secuencia

2.3. Diagrama de componentes de todo el proyecto

3. Diseño de datos

3.1. Diagrama relacional de la BD

3.2. Diccionario de datos

3.2.1. Módulos de usuarios

3.2.2. Módulo de inventario y productos

3.2.3. Módulo de pedidos

3.2.4. Módulo de tablas del sistema

3.3. Listado de cardinalidades de las tablas

3.4. Llaves primarias y foráneas de todas las tablas

4. Interfaces (Mockups)

4.1. Pantalla de inicio de sesión y registro (Cliente externo)

4.2. Catálogo principal de productos (Cliente externo)

4.3. Detalle de producto y verificación de stock

4.4. Resumen de compra y confirmación de pedido (Cliente externo)

4.5. Historial de pedidos y descarga de comprobantes (Cliente externo)

4.6. Inicio de sesión administrativo (Personal interno)

4.7. Panel de control de inventario y gestión de productos

4.8. Gestión logística y cambio de estatus de pedidos

4.9. Tablero de reportes y estadísticas operativas

1. Análisis

1.1 Descripción de la solución planeada

El propósito de este proyecto es resolver los problemas de organización que tiene actualmente la empresa Macuin, ya que sus sistemas actuales no están conectados y generan duplicidad de información. La solución consiste en crear una plataforma web única que conecte a todos los usuarios.

Para lograr esto, se utilizará una arquitectura desacoplada. Esto significa que tendremos un "cerebro" central (una API desarrollada en Python con FastAPI) que controlará toda la información y la guardará en una sola base de datos. De esta forma, nos aseguramos de que los datos sean consistentes y no se repitan.

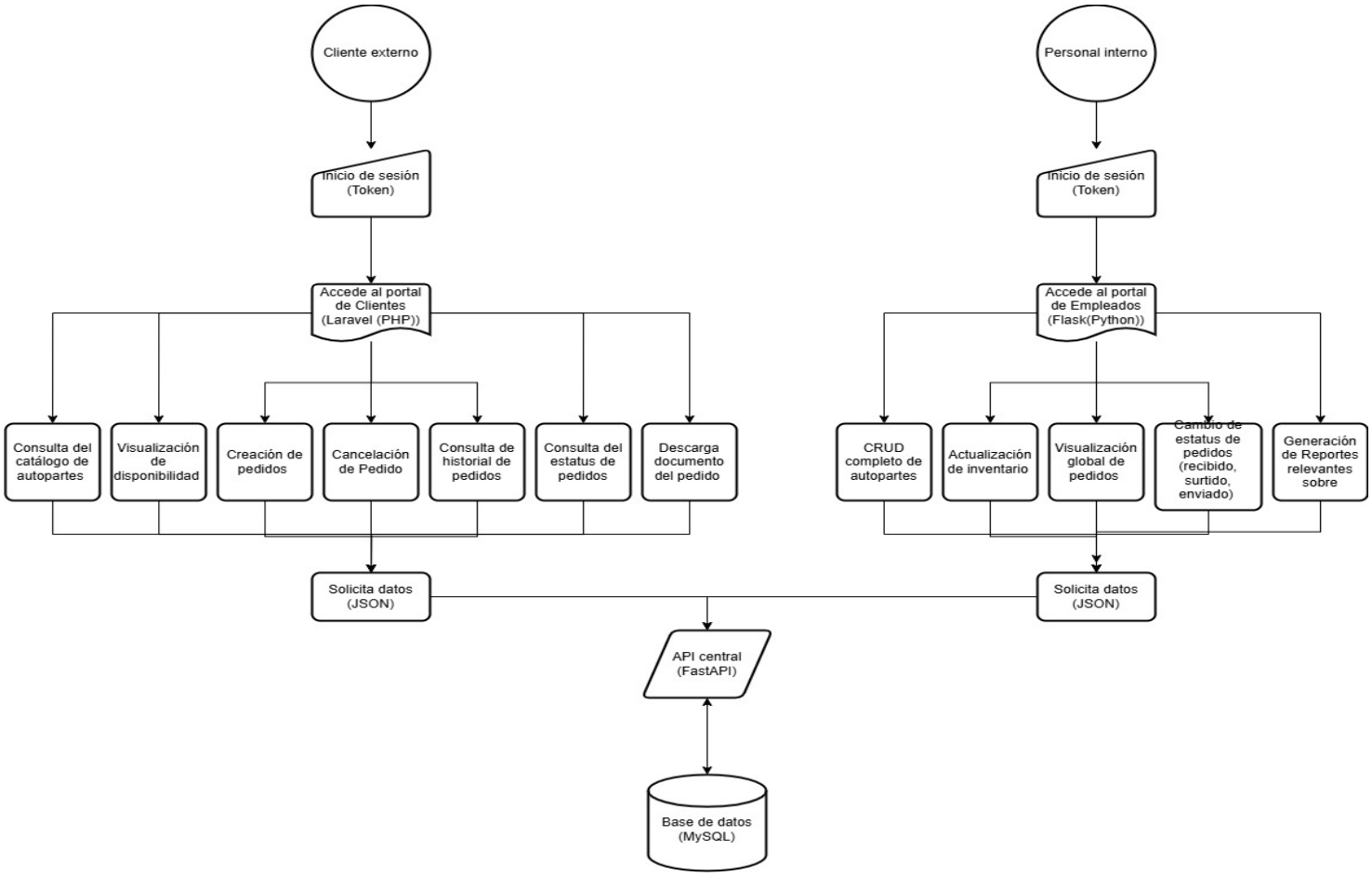
Esta API será consumida por dos páginas web diferentes:

- Portal para clientes: Desarrollado en Laravel (PHP), donde los clientes externos podrán ver el catálogo y hacer sus pedidos de forma sencilla.
- Portal para empleados: Desarrollado en Flask (Python), donde el personal de ventas y almacén podrá administrar el inventario y revisar los pedidos recibidos.

Todo el sistema se comunicará mediante internet usando el formato JSON, asegurando que tanto clientes como empleados vean siempre la misma información en tiempo real.

1.2 Diagrama de flujo de la solución

A continuación, se presenta el diagrama de flujo general de la solución.



Este diagrama de flujo representa de manera general los procesos básicos que estará realizando la solución propuesta. Tenemos dos usuarios principales identificados, Cliente externo y personal interno.

En el primer proceso, el cliente externo entra a su portal exclusivo utilizando una clave de identificación segura. Una vez que inicia sesión, puede realizar las siguientes actividades dentro de la página:

- Consultar el catálogo de autopartes.
- Ver la disponibilidad de piezas en tiempo real.
- Crear nuevos pedidos.
- Cancelar un pedido si es necesario.
- Consultar su historial de compras pasadas.
- Revisar el estatus actual de sus envíos.
- Descargar el documento o comprobante de su compra.

Todas estas funciones trabajan enviando y recibiendo información en un formato de texto estándar, el cual es procesado por nuestro sistema central (FastAPI) para guardar los datos directamente en la base de datos.

El proceso para el personal interno es muy parecido; ellos acceden a su propio portal usando su inicio de sesión y validación de seguridad. Las acciones que pueden realizar los empleados son:

- Administración total de las autopartes (agregar, editar o borrar).
- Actualización manual del inventario.
- Visualización general de todos los pedidos de la tienda.
- Cambio del estatus de los pedidos (para marcar si fue recibido, surtido o enviado).
- Generación de reportes de ventas.

Al igual que con los clientes, la gestión de esta información se realiza mediante el mismo sistema de comunicación centralizado. De esta manera, logramos conectar dos plataformas web construidas con tecnologías diferentes (Laravel y Flask) para que trabajen juntas de forma ordenada.

1.3 Requerimientos funcionales

En la siguiente tabla se detallan los requerimientos funcionales para la interfaz de “Clientes externos”. Estas funciones están diseñadas para permitir a los usuarios registrarse, consultar el catálogo de productos y realizar sus pedidos de manera autónoma.

Requerimientos funcionales				
Interfaz : Cliente externo				
ID	**Nombre**	**Descripción**	**Prioridad**	**Avance**
1	Registro e inicio de sesión	Permitir el registro de usuarios y acceso mediante credenciales.	Alta	0%
2	Consulta de catálogo	Visualizar el listado de autopartes obteniendo datos de la API.	Alta	0%
3	Visualización de stock	Mostrar la disponibilidad de piezas en tiempo real.	Media	0%
4	Creación de pedidos	Permitir seleccionar productos y confirmar la orden de compra.	Alta	0%
5	Historial de pedidos	Listar los pedidos realizados anteriormente por el cliente.	Media	0%
6	Estatus de pedidos	Visualizar el estado actual (surtido, enviado) del pedido.	Alta	0%
7	Cancelación de pedido	Permitir cancelar una orden si no ha sido enviada.	Baja	0%
8	Descarga de documento	Generar y descargar el comprobante del pedido en PDF.	Media	0%

Por otro lado, para la interfaz del “Personal interno” cuenta con funciones específicas de gestión. La siguiente tabla describe los requerimientos necesarios para que los empleados de ventas y almacén puedan administrar el inventario, actualizar el estatus de los pedidos y generar reportes.

Requerimientos funcionales				
Interfaz : Personal interno				
ID	**Nombre**	**Descripción**	**Prioridad**	**Avance**
1	Inicio de sesión	Acceso restringido para personal de ventas y almacén.	Alta	0%
2	CRUD Autopartes	Crear, leer, editar y eliminar información de productos.	Alta	0%
3	Actualización de inventario	Modificar manualmente la cantidad de stock disponible.	Alta	0%
4	Visualización de pedidos	Ver la lista global de pedidos de todos los clientes.	Alta	0%
5	Cambio de estatus	Actualizar estado del pedido a Recibido, Surtido o Enviado.	Alta	0%
6	Reportes de ventas	Generar resumen de ventas y pedidos realizados.	Media	0%
7	Reportes de clientes	Visualizar actividad y datos relevantes de los clientes.	Baja	0%

A continuación, se definen los objetivos de la API Central. Este componente funciona como el núcleo del sistema y es responsable de procesar toda la lógica de negocio, validar la información y gestionar la comunicación segura con la base de datos para responder a las solicitudes de las interfaces web.

Requerimientos funcionales				
Interfaz : API central				
ID	**Requerimiento**	**Descripción**	**Prioridad**	**Avance**
1	Lógica de negocio	Centralizar validaciones y reglas de inventario.	Alta	0%
2	Endpoints REST	Exponer datos en formato JSON para ambos portales.	Alta	0%
3	Gestión de BD	Conexión única y segura a la base de datos central.	Alta	0%

1.4 Requerimientos no funcionales

Por último, se presentan los requerimientos no funcionales que se aplican a las 3 interfaces. Estos puntos establecen las condiciones técnicas y restricciones que el proyecto debe cumplir obligatoriamente, tales como el tipo de arquitectura, los protocolos de comunicación y las herramientas de desarrollo específicas solicitadas para la entrega.

Requerimientos no funcionales				
Interfaz : Cliente externo				
ID	**Nombre**	**Descripción**	**Prioridad**	**Avance**
1	Arquitectura	Uso obligatorio de FastAPI, Laravel y Flask desacoplados.	Alta	0%
2	Comunicación	Intercambio de datos exclusivamente vía JSON/HTTP.	Alta	0%
3	Motores de vista	Uso de Jinja2 (Python) y Blade (PHP) para el frontend.	Media	0%
4	Seguridad	Autenticación mediante Tokens en la API.	Alta	0%
5	Base de Datos	Uso de MySQL/PostgreSQL centralizado.	Alta	0%

2. Diseño arquitectónico

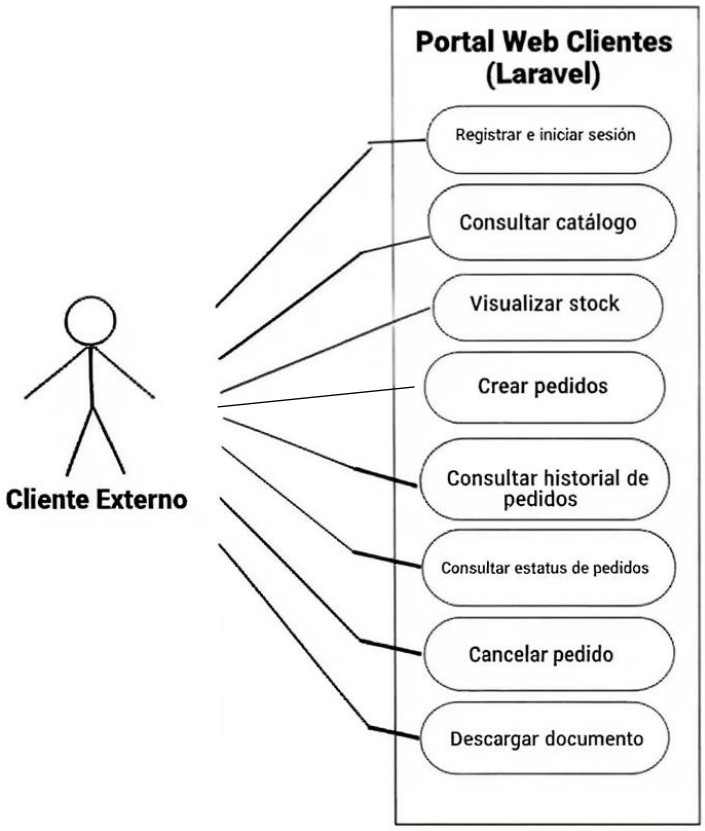
2.1 Diagramas de casos de uso

A continuación, se presentan los diagramas de casos de uso para cada componente del sistema. Estos esquemas definen las interacciones entre los usuarios (actores) y las funciones que ofrece la plataforma, tanto para los clientes externos como para el personal interno y la API central.

Caso de uso (Cliente externo)

En este diagrama se muestran las acciones que puede realizar el cliente externo dentro de la página web pública. Como se puede observar, el usuario tiene acceso a todo el ciclo de compra desde registrarse e iniciar sesión, hasta consultar el catálogo y revisar si hay stock disponible.

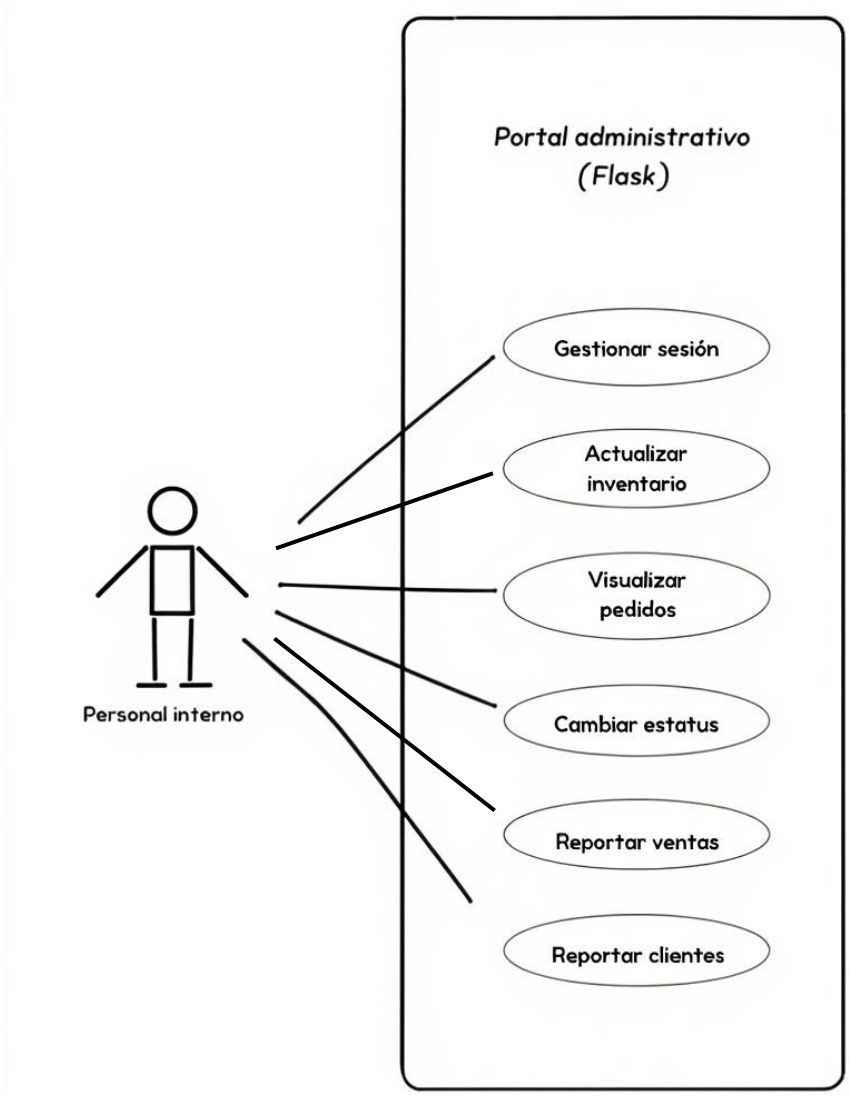
Las funciones principales incluyen la creación de pedidos y el seguimiento de los mismos (ver historial y estatus). Además, se agregó la opción de cancelar pedidos y descargar su comprobante, dándole autonomía al cliente sin tener que llamar a la empresa.



Caso de uso (Personal interno)

Para el personal interno, el sistema funciona diferente. Este diagrama representa el panel de administración donde entran los empleados de ventas y almacén.

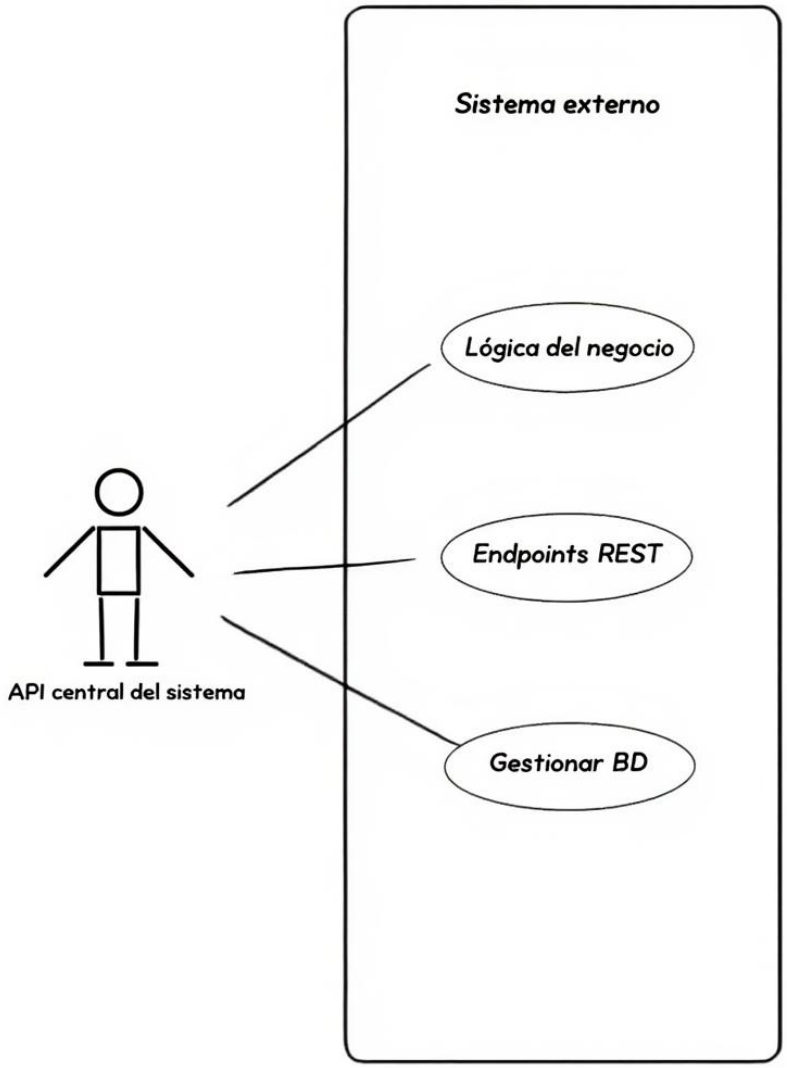
Sus casos de uso están enfocados en el control de la empresa. Lo más importante aquí es la capacidad de actualizar el inventario para mantener los datos reales y la función de cambiar el estatus de los pedidos (por ejemplo, marcar un pedido como "Enviado"). También se incluyen funciones para generar reportes de ventas y clientes, lo cual ayuda a la toma de decisiones.



Caso de uso (API central del sistema)

Finalmente, este diagrama representa la lógica interna del sistema, es decir, el Backend. Aquí el "actor" representa a los sistemas externos (la web de Laravel y el portal de Flask) que consumen los servicios de la API.

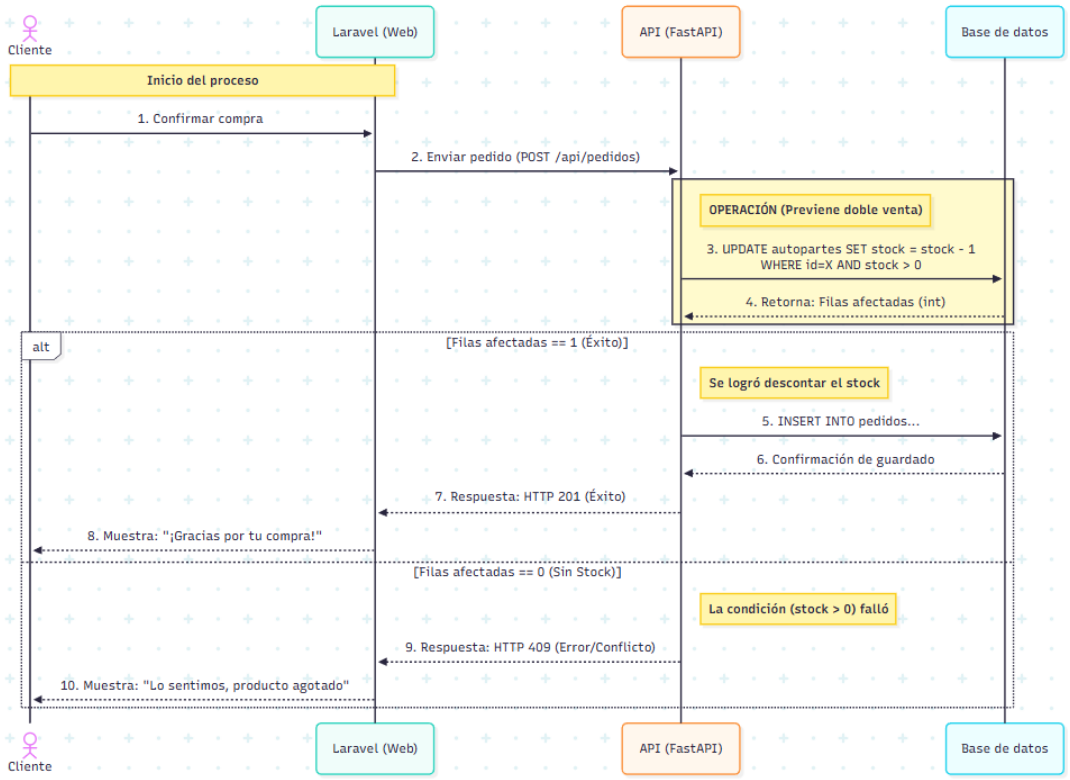
Las funciones de este componente son puramente técnicas y de procesamiento: se encarga de ejecutar la lógica del negocio, exponer los Endpoints REST para que las otras webs se comuniquen, y es el único encargado de gestionar la base de datos de forma segura.



2.2 Diagramas de secuencia

En esta sección se presenta la interacción dinámica entre los componentes del sistema ya que la arquitectura está separada (Frontend separado del Backend), los diagramas detallan cómo viajan las peticiones HTTP desde los clientes web (Laravel/Flask) hacia la API central (FastAPI) y cómo esta última gestiona la conexión en la base de datos relacional.

Diagrama de secuencia de compra (Cliente)



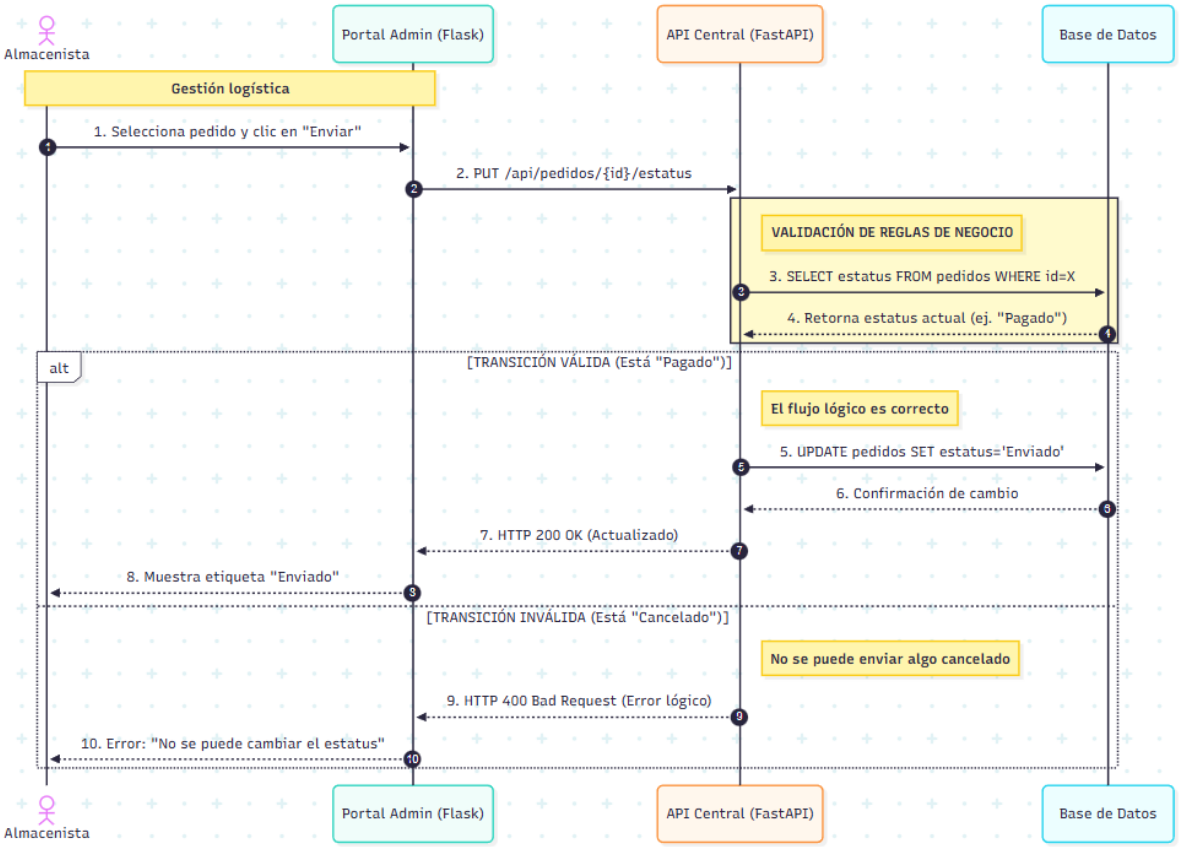
Este diagrama detalla el flujo más importante del sistema: la compra de un producto. Todo el proceso comienza en la interfaz de Laravel, la cual envía la solicitud hacia la API central.

Lo más destacado de este diseño es la lógica de seguridad que aplicamos en la base de datos (paso 3). En lugar de primero leer si hay inventario y luego restarlo, la API intenta descontarlo directamente con una condición lógica (stock mayor a 0).

El bloque "alt" explica las dos decisiones que puede tomar el sistema según el resultado:

- Camino del éxito: Si la base de datos logra restar el stock, significa que el producto sí estaba disponible. En ese momento, el sistema procede a guardar el pedido y confirma la venta al cliente.
- Camino del error: Si la base de datos no pudo restar nada (porque el inventario ya estaba en cero), la operación se detiene de forma segura. El sistema detecta esto y devuelve un mensaje de error, evitando así que vendamos productos que no existen.

Diagrama de secuencia de cambio de estatus (personal interno)



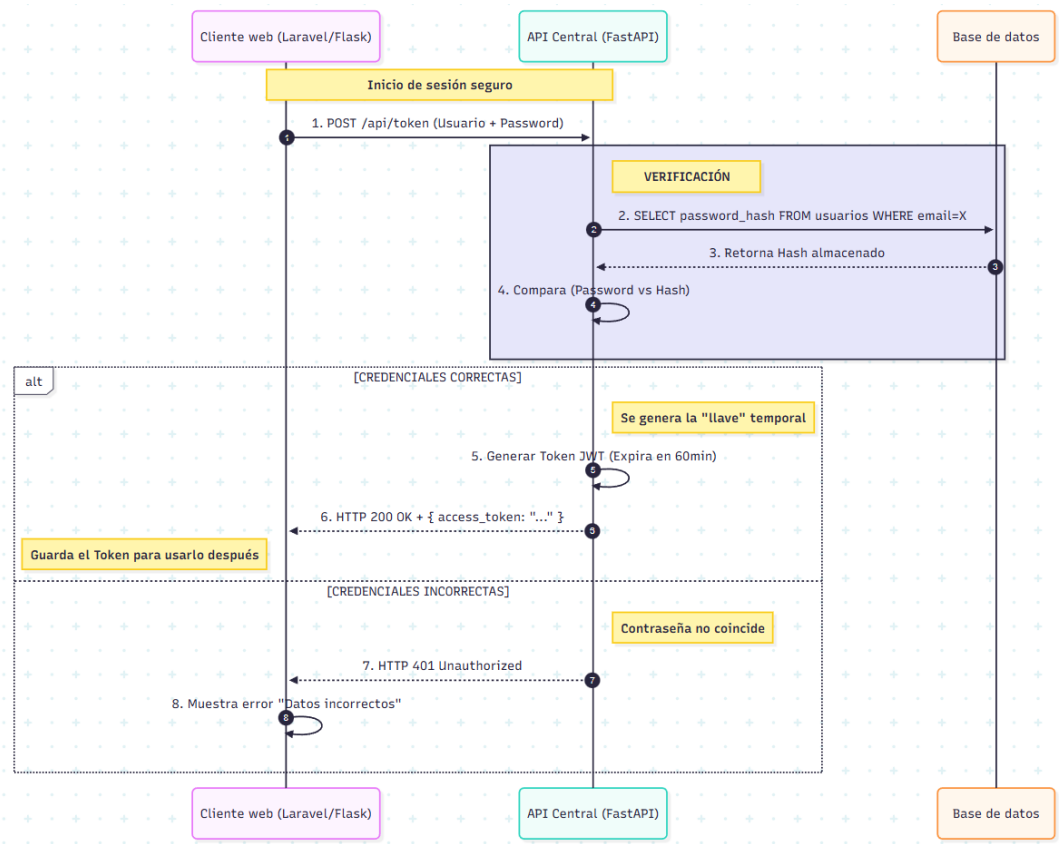
Este diagrama muestra el flujo de trabajo del personal interno mediante el portal administrativo (Flask). El objetivo principal es actualizar el estado de un pedido para avanzar en la logística, por ejemplo, cambiando de "Pagado" a "Enviado".

Lo más relevante de este proceso es la validación que realiza la API central (pasos 3 y 4). Antes de guardar cualquier cambio, el sistema verifica primero en la base de datos cuál es el estado actual del pedido.

El bloque "alt" explica las reglas de protección que aplicamos:

- Transición válida: Si el pedido está en un estado correcto (como "Pagado"), la API permite la actualización y registra la fecha de envío.
- Transición inválida: Si el pedido tiene un estado incompatible (como "Cancelado" o "Enviado"), la API rechaza la solicitud con un error. Esto ayuda a evitar inconsistencias o confusiones en el almacén.

Diagrama de secuencia de autenticación y seguridad



Este diagrama detalla el mecanismo de seguridad que cubre todo el sistema. Como la arquitectura no guarda sesiones activas, la API no recuerda a los usuarios entre cada petición. Por ello, es obligatorio obtener un token de acceso al principio.

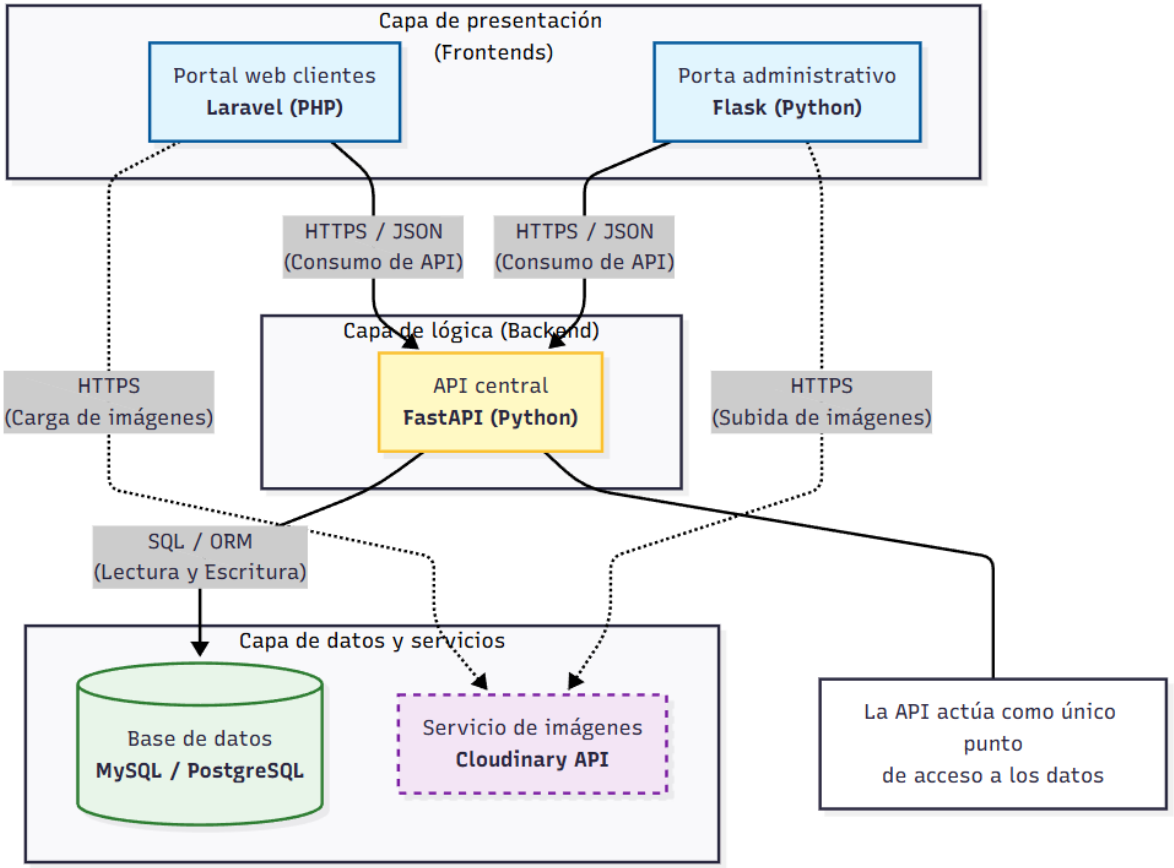
El flujo muestra el intercambio de información:

- El cliente web envía el usuario y la contraseña una sola vez.
- La API consulta la contraseña encriptada en la base de datos y la compara.

El bloque condicional explica el resultado de esta comparación:

- Credenciales correctas: si coinciden, la API genera un token que servirá como llave para las siguientes peticiones y se lo devuelve al cliente.
- Credenciales incorrectas: si fallan, se niega el acceso con un error, protegiendo así los datos del sistema contra intrusos.

2.3 Diagrama de componentes de todo el proyecto



En este esquema mostramos cómo organizamos las partes del sistema para que funcionen de manera ordenada. Decidimos separar el proyecto en tres capas principales (Presentación, lógica y datos) para que el código no se mezcle y sea más fácil de mantener.

Como se ve en la parte superior, tenemos dos aplicaciones web independientes:

- Portal web clientes (Laravel): Es la página pública donde entran los compradores.
- Portal administrativo (Flask): Es el panel interno para los empleados.

Lo importante de nuestro diseño es que ninguno de estos dos portales se conecta directamente a la base de datos. Ambos funcionan solo como "vistas" que piden información a la capa del medio.

En el centro tenemos la API central (FastAPI). Esta es la parte más importante del proyecto porque actúa como un "filtro de seguridad". Es la única que tiene permiso para leer o guardar información. Si Laravel o Flask necesitan algo, se lo tienen que pedir a la API mediante mensajes JSON.

Por último, en la capa de abajo están nuestros datos. Usamos una base de datos relacional para guardar los pedidos y el inventario. Además, agregamos un servicio externo llamado Cloudinary, que sirve para guardar las fotos de las autopartes en la nube, esto ayuda a que nuestro servidor principal no se vuelva lento por cargar muchas imágenes pesadas.

Diseño de datos

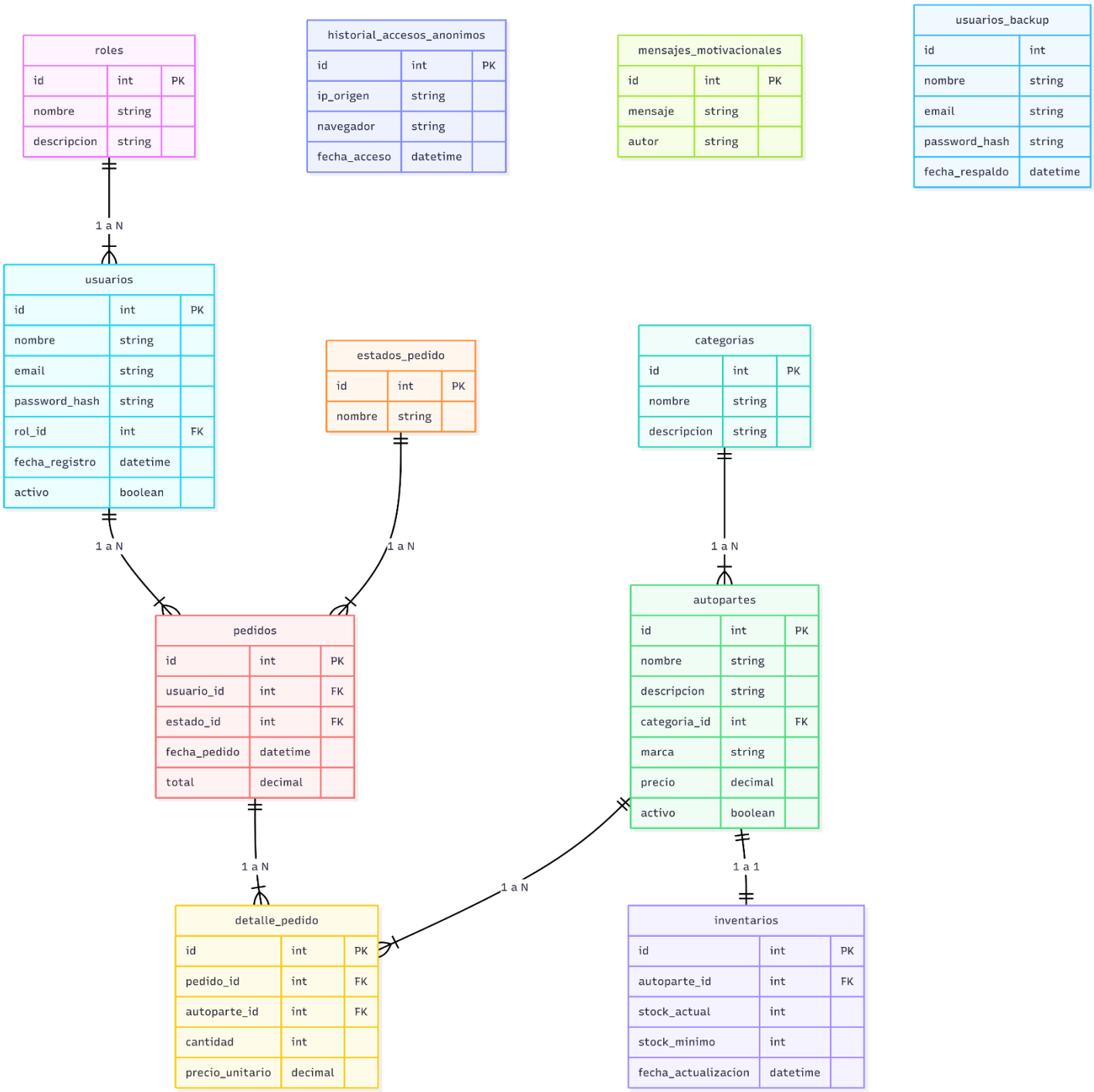
3.1 Diagrama relacional de la BD

En esta sección presentamos la estructura visual de nuestra base de datos autopartes_db. Para que el esquema sea legible, lo hemos dividido en dos partes, el núcleo del negocio (donde ocurren las ventas) y las tablas de soporte.

1. Entidades principales y relaciones

En la primera imagen se observa el flujo central del sistema. Todo está diseñado para mantener la integridad de la información:

- En la parte de usuarios y roles, un usuario pertenece a un rol específico (1 a N), lo que nos permite diferenciar entre clientes y personal administrativo.
- El flujo de venta sirve gracias a la tabla central es pedidos. Esta tabla conecta al usuario que compra con el estado actual de su envío. Sin embargo, para saber qué compró, utilizamos la tabla intermedia detalle_pedido.
- Un punto importante de este diseño es que separamos la información de la pieza (autopartes) de su cantidad física (inventarios). Tienen una relación de 1 a 1, lo que significa que cada pieza tiene su propio registro único de stock, ayudándonos a evitar errores de conteo.


roles
id int PK
nombre string
descripcion string
historial_accesos_anonimos
id int PK
ip_origen string
navegador string
fecha_acceso datetime
mensajes_motivacionales
id int PK
mensaje string
autor string
usuarios_backup
id int
nombre string
email string
password_hash string
fecha_respaldo datetime
1 a N
usuarios
id int PK
nombre string
email string
password_hash string
rol_id int FK
fecha_registro datetime
activo boolean
estados_pedido
id int PK
nombre string
categorias
id int PK
nombre string
descripcion string
1 a N
1 a N
1 a N
pedidos
id int PK
usuario_id int FK
estado_id int FK
fecha_pedido datetime
total decimal
autopartes
id int PK
nombre string
descripcion string
categoria_id int FK
marca string
precio decimal
activo boolean
1 a N
1 a N
1 a 1
detalle_pedido
id int PK
pedido_id int FK
autoparte_id int FK
cantidad int
precio_unitario decimal
inventarios
id int PK
autoparte_id int FK
stock_actual int
stock_minimo int
fecha_actualizacion datetime

Además de las relaciones principales, el sistema utiliza tablas independientes para funciones técnicas que no requieren estar "amarradas" a una venta. Como se ve en la siguiente imagen, tenemos tablas para guardar logs generales (historial de eventos), configuraciones del sistema y backups de usuarios. También incluimos tablas temporales como autopartes_temp, que sirven para operaciones intermedias sin afectar la base de datos principal.

autopartes_temp		
id	int	PK
nombre	string	
precio	decimal	

estado_pedido_texto		
id	int	PK
pedido_id	int	
estado	string	

pedidos_resumen		
pedido_id	int	PK
total_productos	int	
total_pedido	decimal	

logs_generales		
id	int	PK
descripcion	string	
fecha	datetime	

configuracion_sistema		
id	int	PK
clave	string	
valor	string	

3.2 Diccionario de datos

En esta parte, se describen detalladamente las tablas que componen la base de datos, especificando el tipo de dato y la función de cada campo dentro del sistema.

3.2.1 Módulos de usuarios

- **Usuarios:** En esta tabla se almacena la información de las personas que acceden al sistema, tomando en cuenta tanto a empleados como clientes.

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador único del usuario.
nombre	VARCHAR(100)		Nombre completo de la persona.
email	VARCHAR(100)		Correo para iniciar sesión (es único).
password_hash	VARCHAR(100)		Contraseña cifrada (no se guarda en texto plano).
rol_id	INT	FK	Relación con la tabla roles para saber sus permisos.
fecha_registro	DATETIME		Fecha en la que se creó la cuenta.
activo	BOOLEAN		Define si el usuario puede entrar (1) o está bloqueado (0).

- **Roles:** En esta tabla se define qué tipos de perfil existen en el sistema.

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador del rol.
nombre	VARCHAR(50)		Nombre del perfil ("Cliente", "Admin").
descripcion	VARCHAR(150)		Explicación breve de lo que puede hacer ese rol.

- **Usuarios_backup:** Esta tabla es importante ya que aquí se guarda una copia de seguridad de los datos de los usuarios antes de borrar/modificar usuarios.

Campo	Tipo	Llave	Descripción
id	INT		El ID original que tenía el usuario.
nombre	VARCHAR(100)		Nombre respaldado.
email	VARCHAR(100)		Email respaldado.
fecha_respaldo	DATETIME		Fecha en que se hizo la copia.

3.2.2 Módulo de inventario y productos

- **Autopartes:** Esta tabla contiene la información de los productos.

Campo	Tipo	Llave	Descripción
id	INT	PK	Código único de la pieza.
nombre	VARCHAR(150)		Nombre del producto.
descripcion	TEXT		Detalles técnicos de la autoparte.
categoria_id	INT	FK	Relación con la tabla “categorías” para clasificar la pieza.
marca	VARCHAR(100)		Fabricante de la pieza.
precio	DECIMAL		Costo unitario de venta.
activo	BOOLEAN		Si está en true, la pieza es visible en la web.

- **Inventarios:** En esta tabla se maneja la existencia de los productos.

Campo	Tipo	Llave	Descripción
id	INT	PK	ID del registro de inventario.
autoparte_id	INT	FK	ID del producto asociado (Relación 1 a 1).
stock_actual	INT		Cantidad real que tenemos en almacén.
stock_minimo	INT		Cantidad límite para saber cuándo resurti.
fecha_actualizacion	DATETIME		Última vez que cambió la cantidad.

- **Categorías:** En esta tabla se organizan las autopartes en grupos.

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador de la categoría.
nombre	VARCHAR(100)		Nombre de la categoría como por ejemplo, "Frenos").
descripcion	VARCHAR(150)		Explicación de qué piezas van aquí.

3.2.3 Módulo de pedidos

- **Pedidos:** En esta tabla se guarda la información general de cada venta que se realiza.

Campo	Tipo	Llave	Descripción
id	INT	PK	Folio del pedido.
usuario_id	INT	FK	Cliente que hizo la compra.
estado_id	INT	FK	Estatus actual del envío (como por ejemplo, "En camino").
fecha_pedido	DATETIME		Fecha y hora de la compra.
total	DECIMAL		Monto total a pagar.

- **Detalle_pedido:** Especifica qué productos y cuántas piezas lleva cada pedido.

Campo	Tipo	Llave	Descripción
id	INT	PK	ID del renglón de detalle.
pedido_id	INT	FK	Folio del pedido al que pertenece.
autoparte_id	INT	FK	Producto que se está comprando.
cantidad	INT		Número de piezas de ese producto.
precio_unitario	DECIMAL		Precio al momento de la compra (por si cambia después).

- **Estados_pedido:** En esta tabla se muestran los estados del pedido.

Campo	**Tipo**	**Llave**	**Descripción**
id	INT	PK	Identificador del estado.
nombre	VARCHAR(50)		Descripción (CREADO, PAGADO, ENVIADO, ETC).

3.2.4 Módulo de tablas del sistema

- Configuración_sistema: En esta tabla se almacenan variables globales para no tener que modificar el código fuente si algo cambia.

Campo	Tipo	Llave	Descripción
id	INT	PK	Identificador.
clave	VARCHAR(100)		Nombre de la configuración (Como por ejemplo, “iva”).
valor	VARCHAR(100)		Valor asignado a esa clave.

- Logs_generales: En esta tabla se guardan los registros como los eventos “importantes” o algún error que se presente en el servidor.

Campo	Tipo	Llave	Descripción
id	INT	PK	ID del evento.
descripcion	TEXT		Qué fue lo que pasó (ej. "Error al pagar").
fecha	DATETIME		Cuando ocurrió el evento.

3.3 Listado de cardinalidades de las tablas

En esta parte se detallan las reglas de multiplicidad que rigen la base de datos autopartes_db. Estas relaciones aseguran que la información no se duplique y mantenga su integridad referencial.

Entidad origen (Padre)	Cardinalidad	Entidad destino (Hija)	Descripción de la regla de negocio
roles	1 a N	usuarios	Un rol (como, por ejemplo, "Cliente") puede estar asignado a muchos usuarios, pero un usuario solo puede tener un rol a la vez.
categorias	1 a N	autopartes	Una categoría (como, por ejemplo, "Frenos") agrupa muchas autopartes diferentes.
autopartes	1 a 1	inventarios	Caso especial: Cada autoparte tiene un único registro de inventario asociado. No puede haber dos registros de stock para la misma pieza (restricción UNIQUE).
usuarios	1 a N	pedidos	Un usuario puede realizar muchos pedidos a lo largo del tiempo.
estados_pedido	1 a N	pedidos	Un estado (como, por ejemplo, "Enviado") puede etiquetar a muchos pedidos simultáneamente.
pedidos	1 a N	detalle_pedido	Un pedido se compone de uno o muchos renglones de detalle (cada renglón es un producto distinto).
autopartes	1 a N	detalle_pedido	Una misma autoparte puede aparecer vendida en muchos detalles de diferentes pedidos.

3.4 Llaves primarias y foráneas de todas las tablas

Tabla	Llave primaria (PK)	Llaves foráneas (FK) y referencia
roles	id	No aplica
usuarios	id	rol_id - Referencia a roles(id)
categorias	id	No aplica
autopartes	id	categoria_id - Referencia a categorias(id)
inventarios	id	autoparte_id - Referencia a autopartes(id)
estados_pedido	id	No aplica
pedidos	id	1. usuario_id - Referencia a usuarios(id) 2. estado_id - Referencia a estados_pedido(id)
detalle_pedido	id	1. pedido_id - Referencia a pedidos(id) 2. autoparte_id - Referencia a autopartes(id)
historial_accesos	id	No aplica (Tabla de bitácora)
mensajes_motivacionales	id	No aplica
logs_generales	id	No aplica
configuracion_sistema	id	No aplica
usuarios_backup	Sin PK definida	No aplica (Tabla de respaldo plana)

Interfaces

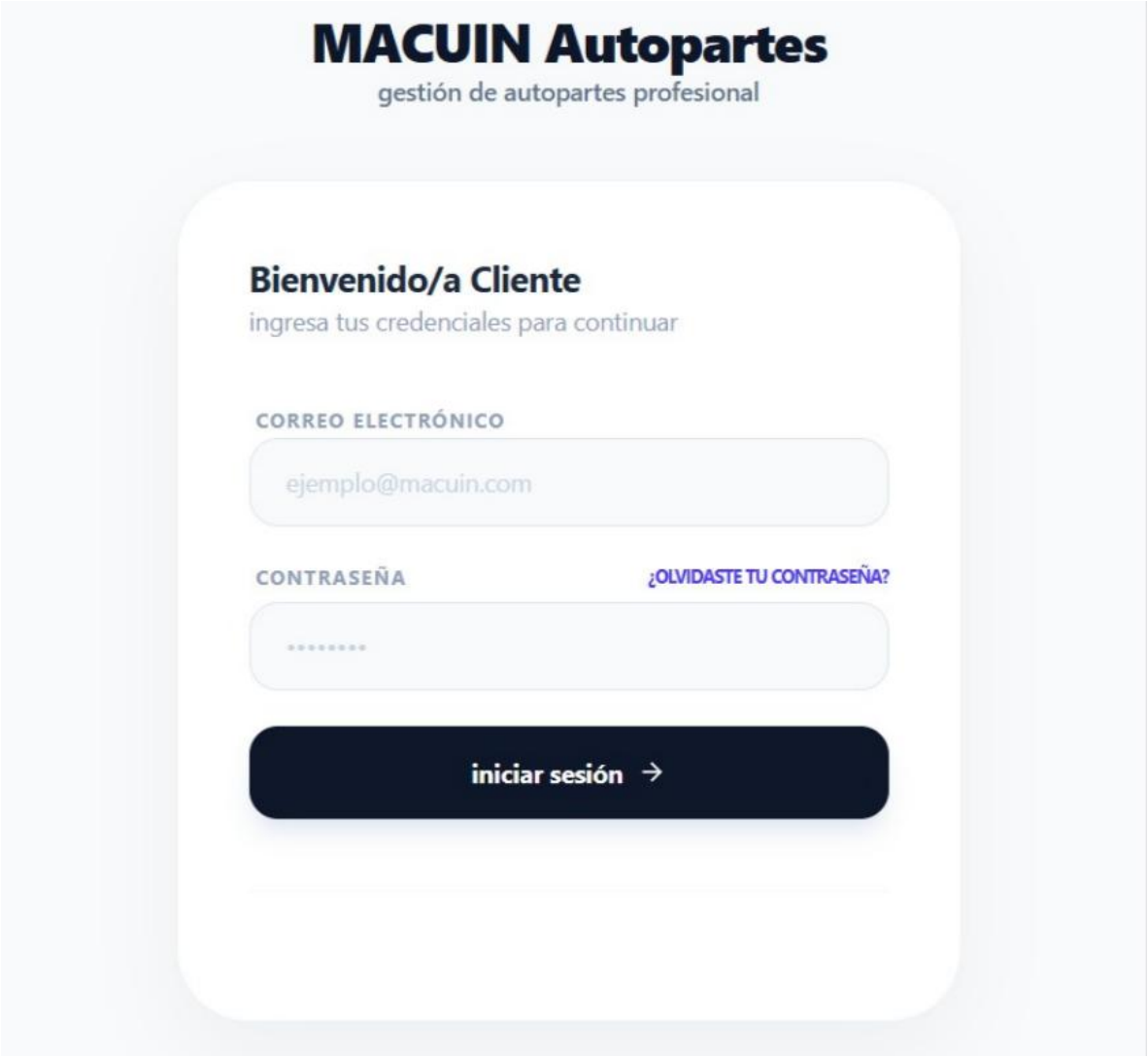
En esta parte, realizamos los mockups diseñados para la plataforma. El desarrollo de estas interfaces obedece a la separación de roles establecida en la arquitectura del sistema, dividiendo la experiencia visual en dos entornos distintos según el tipo de usuario.

Los objetivos del diseño es cumplir con lo siguiente:

1. Portal de clientes (Entorno Laravel): Una interfaz intuitiva, siguiendo los patrones de diseño comunes en tiendas en línea (e-commerce). El objetivo es reducir la curva de aprendizaje para que cualquier usuario pueda buscar refacciones y realizar compras sin asistencia.
2. Portal administrativo (Entorno Flask): Para el área operativa, priorizamos la densidad de información y la rapidez de gestión. El diseño se basa en tablas de datos y paneles de control (dashboards) que permiten a los empleados administrar el inventario y el flujo de pedidos de manera eficiente.

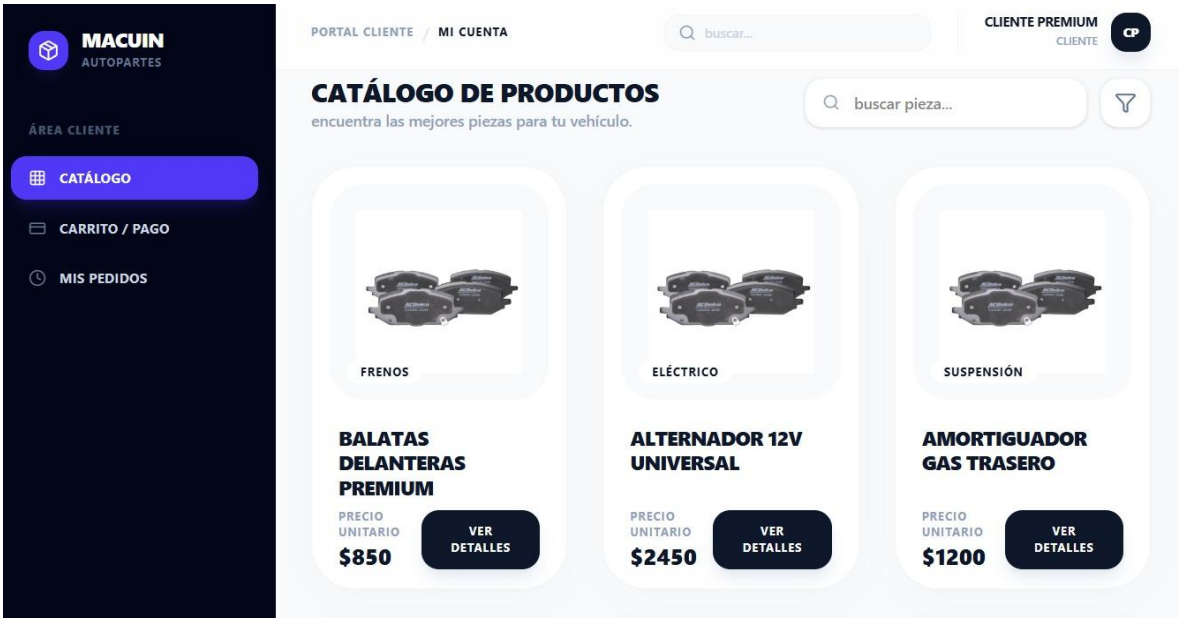
A continuación, se detallan las interfaces propuestas para cada módulo.

4.1 Pantalla de inicio de sesión y registro (Cliente externo)



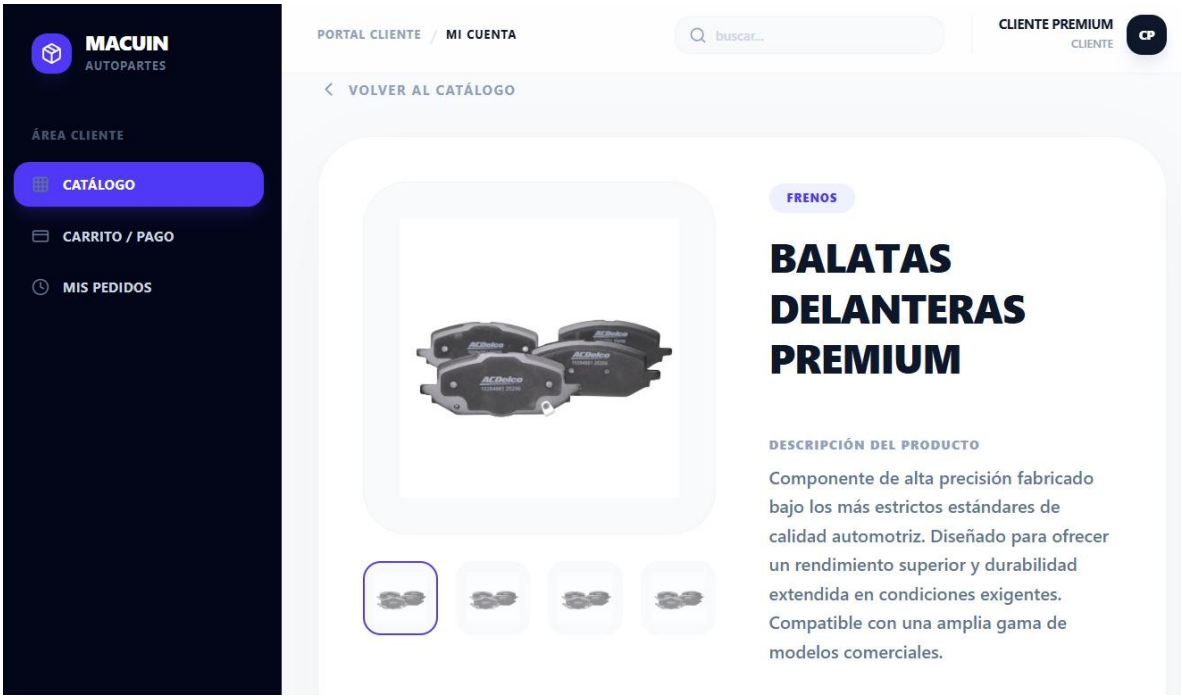
En esta interfaz se presenta el acceso principal para los usuarios que deseen comprar autopartes. El diseño se mantuvo minimalista para facilitar su uso, solicitando únicamente el correo electrónico y la contraseña para ingresar al portal. Además, se integró la opción de registro en la misma pantalla mediante el enlace inferior, permitiendo que los nuevos clientes puedan crear su cuenta rápidamente sin tener que navegar a otra sección, cumpliendo así con el requerimiento de autenticación de la plataforma.

4.2 Catálogo principal de productos (Cliente externo)



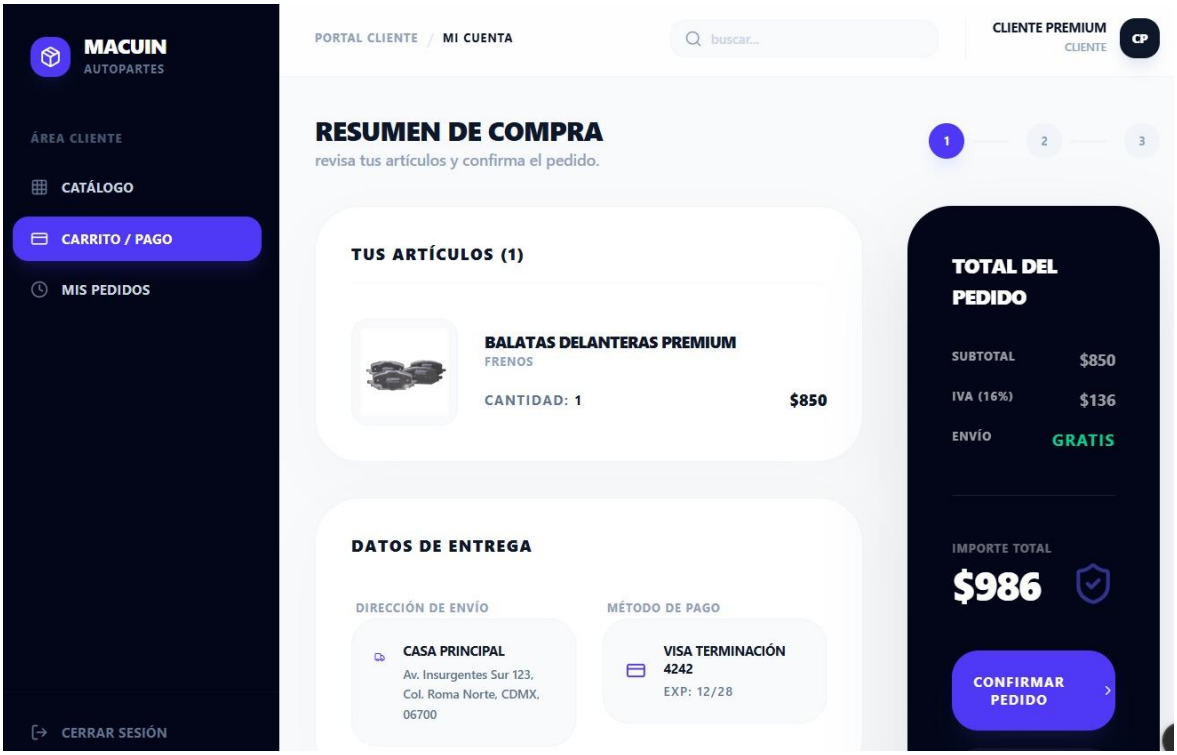
Esta interfaz fue diseñada para que el usuario encuentre rápidamente lo que busca. En la parte superior se ubica una barra de búsqueda global y el acceso al carrito de compras. El cuerpo principal organiza las autopartes en un diseño de cuadrícula (grid) con tarjetas individuales que muestran la foto, el nombre de la categoría o pieza y botones de acción directa. Además, se incluye una sección lateral de "Filtros" para segmentar la búsqueda por categoría.

4.3 Detalle de producto y verificación de stock



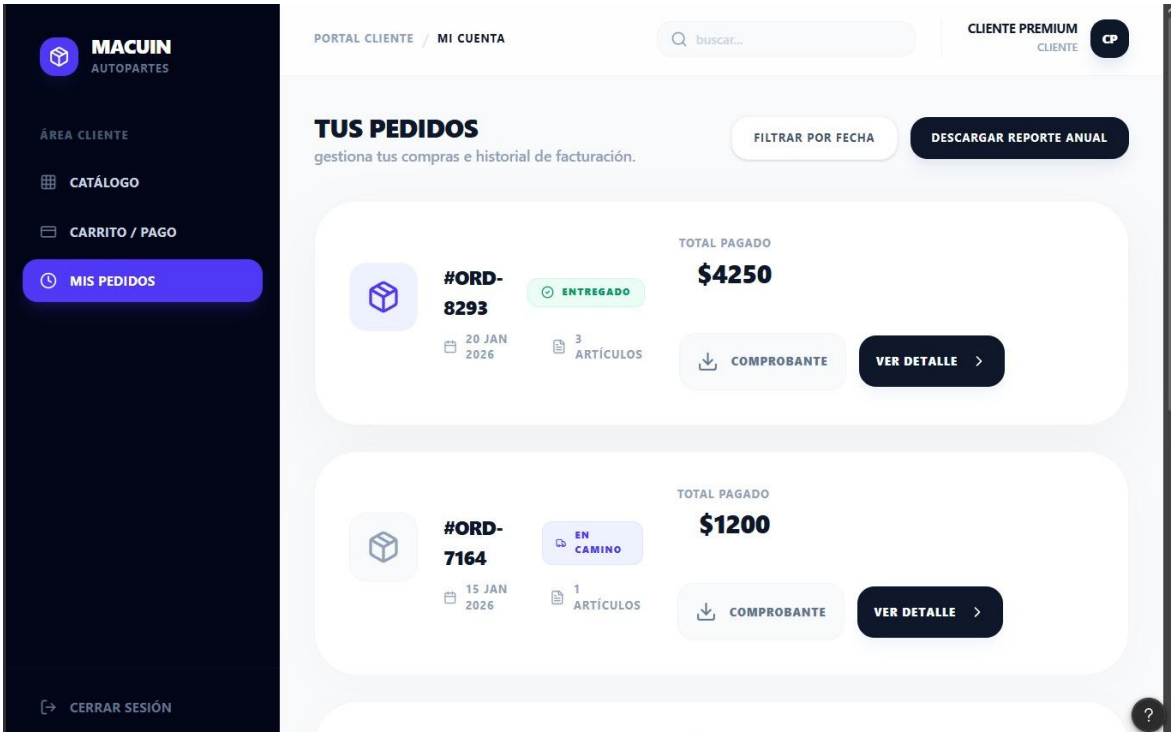
Esta interfaz permite al usuario consultar las especificaciones técnicas de una autoparte antes de adquirirla. Elemento clave de esta pantalla es la visualización de disponibilidad en tiempo real, la cual informa al cliente el número exacto de piezas en existencia. Esta función es fundamental para la experiencia de usuario, ya que previene la selección de artículos agotados y asegura la transparencia del inventario antes de proceder al carrito de compras.

4.4 Resumen de compra y confirmación de pedido (Cliente externo)



Esta interfaz representa la etapa final del proceso de adquisición, el diseño permite al usuario revisar detalladamente los artículos seleccionados, modificar cantidades o eliminar productos antes de proceder al pago. Un componente esencial es el panel de desglose financiero, que muestra de forma transparente el subtotal, el cálculo de impuestos (IVA) y el monto final. El botón de "Confirmar y pagar" es el que activa el proceso transaccional, enviando la solicitud formal a la API para registrar el pedido y apartar el inventario.

4.5 Historial de pedidos y descarga de comprobantes (Cliente externo)

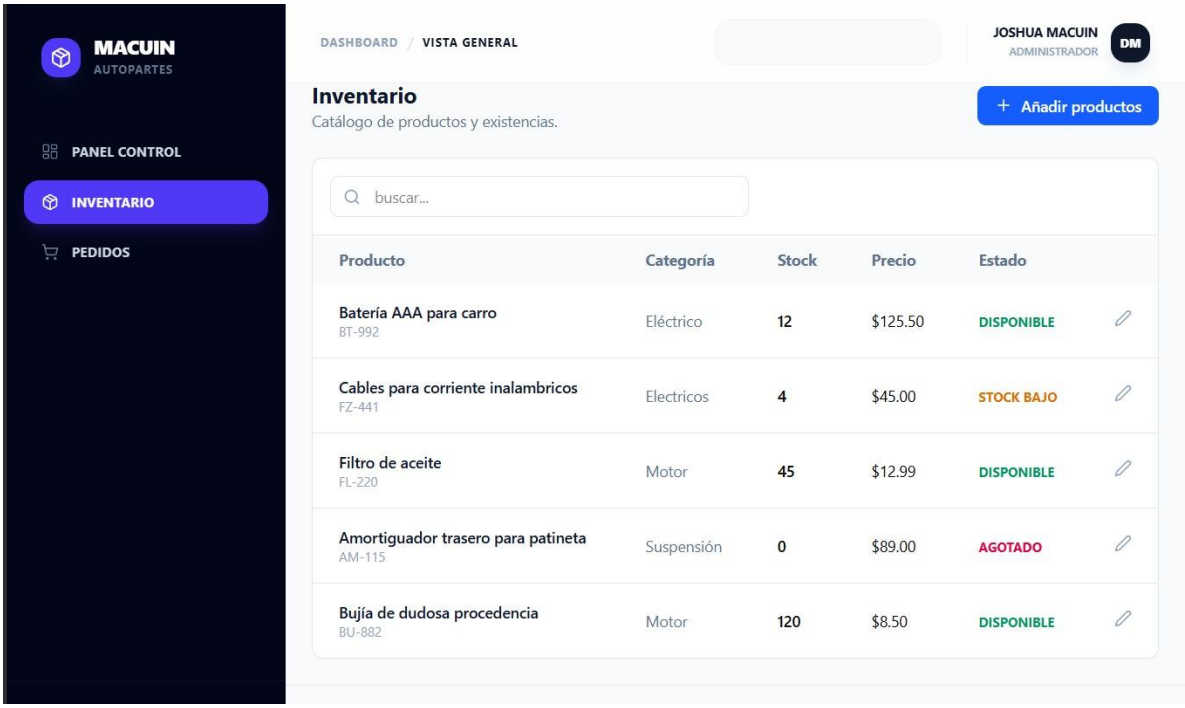


Esta interfaz está diseñada para brindar autonomía al usuario en la gestión post-venta. A través de un listado cronológico, el cliente puede monitorear el ciclo de vida de sus compras, visualizando información como el folio de rastreo y el monto total. La columna de "Estatus" ofrece retroalimentación inmediata sobre el estado logístico del paquete (si ha sido surtido o enviado). Además, se incorporaron en la sección de acciones los botones para consultar el detalle de la orden y descargar el comprobante oficial en formato PDF.

4.6 Inicio de sesión administrativo (Personal interno)

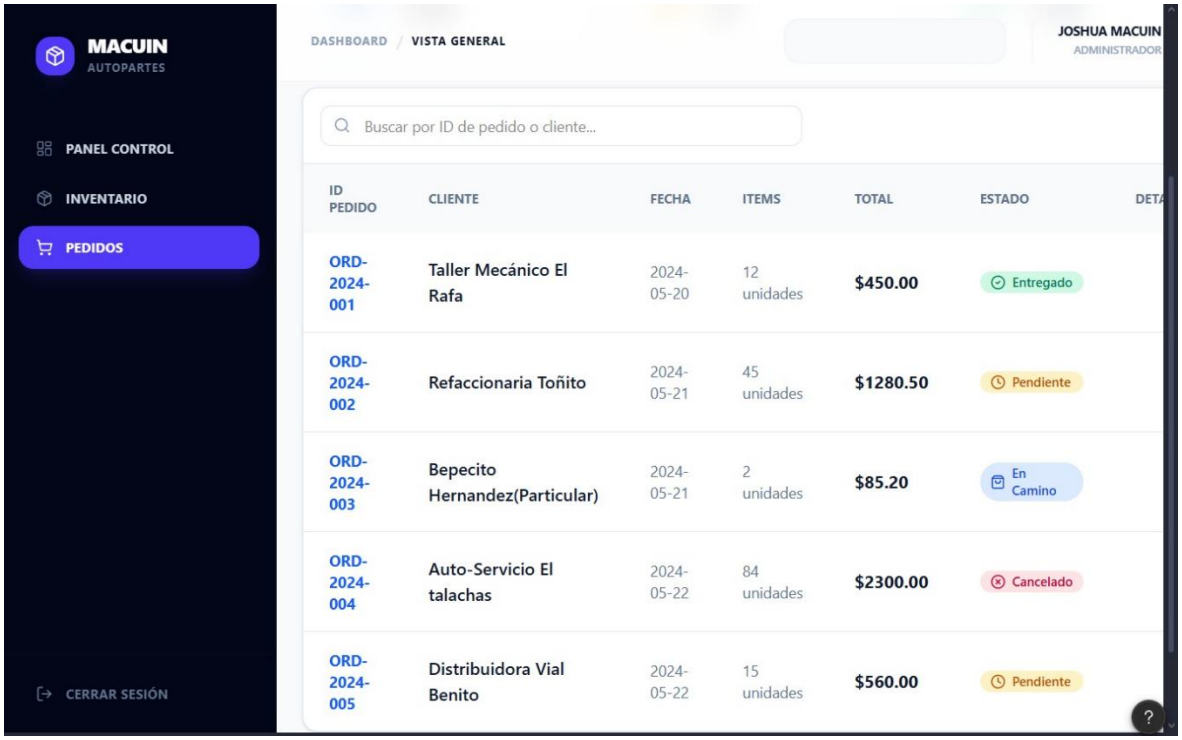
El acceso al portal de gestión está restringido exclusivamente al personal autorizado de ventas y almacén. A diferencia de la interfaz pública, este módulo no permite el auto-registro, las credenciales son suministradas directamente por el administrador del sistema. El diseño prioriza la seguridad, solicita un identificador de empleado único y contraseña para validar los permisos correspondientes mediante la API, garantizando así que solo los roles adecuados puedan modificar información sensible como inventarios o estatus de pedidos.

4.7 Panel de control de inventario y gestión de productos (Personal interno)



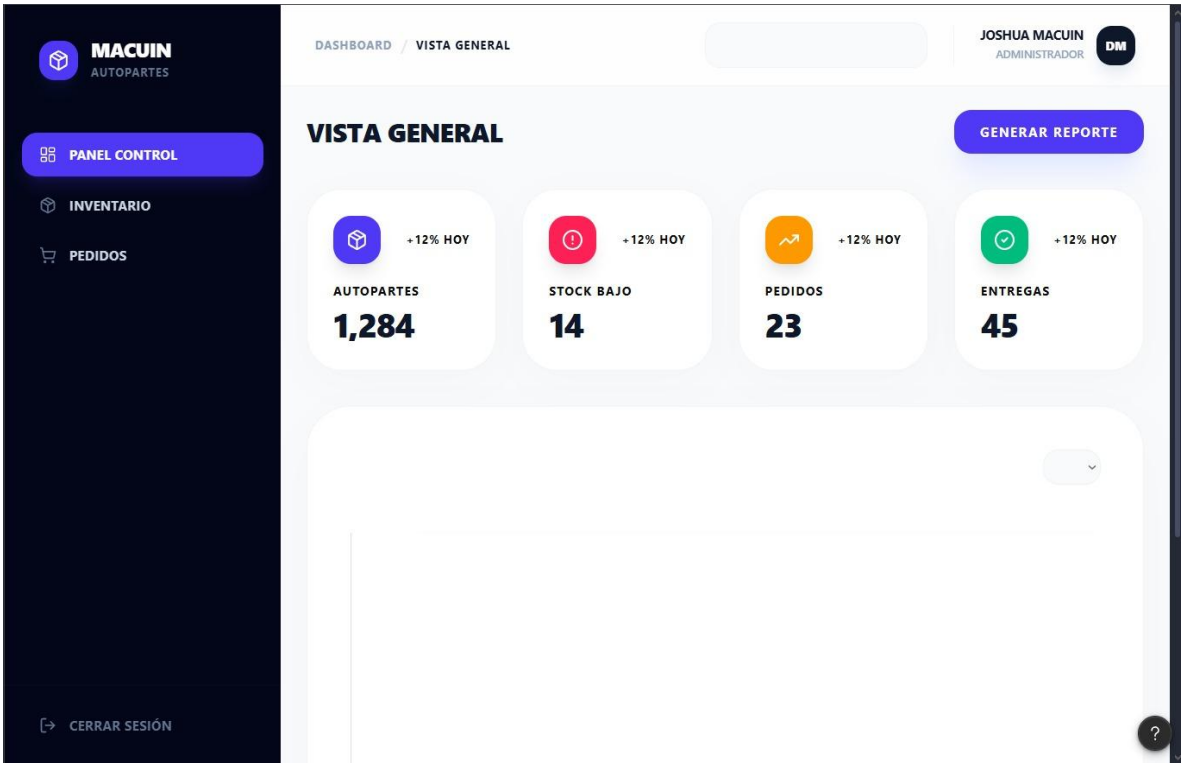
Esta interfaz funge como el centro de operaciones para el personal de almacén, es un dashboard el cual permite la administración eficiente del catálogo de autopartes. La tabla central ofrece una vista rápida del stock actual, resaltando visualmente aquellos productos con existencias críticas (como se observa en el ítem con stock "0"). También incluye herramientas de edición y eliminación incorporadas en cada fila, junto con el botón superior de “Nueva”, las cuales completan el ciclo CRUD, otorgando al empleado control total sobre la base de datos de productos.

4.8 Gestión logística y cambio de estatus de pedidos (Personal interno)



Esta interfaz es fundamental para el flujo de trabajo del almacén, ya que, a través de ella, el personal visualiza la lista de órdenes recibidas, en la cual se muestra al cliente y la fecha de solicitud. La columna "Status" es interactiva, incorporando un menú desplegable que permite al personal actualizar el estado del pedido en tiempo real (de "Recibido" a "En proceso" y finalmente a "Enviado"). Esta acción dispara la actualización en la base de datos central y notifica al cliente, cerrando el ciclo de venta.

4.9 Tablero de reportes y estadísticas operativas (Personal interno)



Esta interfaz la diseñamos para que proporcione datos de negocios básicos para la toma de decisiones, ya que, el sistema procesa los datos históricos de ventas y actividad de usuarios para generar visualizaciones gráficas y métricas clave (KPIs) en tiempo real, como el volumen de ventas diario y el conteo de nuevos pedidos. Esta herramienta permite a los supervisores de ventas y almacén monitorear el desempeño general de la empresa y detectar tendencias sin necesidad de realizar cálculos manuales o consultar directamente la base de datos.